Ira A. Fulton Schools of Engineering
Arizona State University

CSE 531: Distributed and Multiprocessor Operating Systems

Client-Centric Consistency Written Report

Problem Statement

A replicated banking system can return stale data when a customer changes branches. For example, a customer may deposit money at Branch 1 and then query their balance through Branch 2 before the deposit has naturally propagated. If Branch 2 has not received the deposit yet, it could incorrectly return the old balance. This violates the customer's expectation that a successful update they performed remains visible in later accesses.

The problem is therefore to extend the original gRPC banking system with client-centric consistency, specifically the Read Your Writes policy. The system must ensure that each customer observes the effects of that customer's own successful deposits and withdrawals during later requests, even after moving to a different branch.

Goal

The goal is to ensure that the effect of every successful customer write is visible to that same customer's later reads and updates, regardless of which branch receives the request.

The implementation accomplishes this by having branches assign unique write IDs to successful deposits and withdrawals. Customers retain the IDs of their completed writes in a writeset and include that writeset in every later request. When a branch receives a request, it compares the customer's writeset with its local write log. If writes are missing, the branch retrieves and applies them before processing the request. This prevents stale query results and ensures that later withdrawals or deposits operate on a state that includes the customer's prior writes.

Setup

The project was implemented in Python using the gRPC banking project as the baseline. The application consists of one client process, multiple branch-server processes, customer logic, branch logic, generated protobuf files, and an input.json file describing customer events and initial branch balances.

The following technologies and versions were used:

1. Python 3.13 virtual environment for local development and testing
2. grpcio==1.71.2
3. grpcio-tools==1.71.2
4. protobuf==5.29.6
5. Git/GitHub for source control
6. Protocol Buffers (protos/banks.proto) to define the gRPC request, response, and service interfaces

The protobuf files were generated with:

```
python -m grpc_tools.protoc \
    -Iprotos \
     --python_out=. \
     --grpc_python_out=. \
      protos/banks.proto
```

The branch servers and client were executed using the required project commands:

python server.py input.json
python client.py input.json

The final project contains the required files: server.py, client.py, customer.py, branch.py, input.json, output.json, and protos/banks.proto.

Implementation Processes

Write Identification and Writesets

Each branch generates a unique ID for every original successful deposit or withdrawal. The ID combines the branch ID and a local write counter, such as 1:4, meaning the fourth write created by Branch 1. This prevents separate branches from generating conflicting write IDs.

Each customer maintains a writeset containing the IDs of its successful writes. The customer includes its current writeset with every query, deposit, and withdrawal request. This gives the receiving branch the information needed to determine whether it has already applied all writes that the customer must be able to observe.

Customer Request Routing

Unlike the baseline gRPC project, a customer is allowed to send each event to a different branch. The customer selects the branch identified by the event's branch field and sends the operation, amount when applicable, and its current writeset.

When a deposit or withdrawal succeeds, the customer adds the returned write ID to its writeset. Query events do not create write IDs because they do not modify the replicated state.

Branch Synchronization

Each branch maintains:

7. Its current bank balance
8. A local write log mapping write IDs to the associated operation and amount
9. A local counter for generating original write IDs
10. gRPC stubs for communicating with peer branches

Before processing a customer query, deposit, or withdrawal, the branch compares the customer's writeset with the keys in its local write log. The missing writes are calculated as:

missing_writes = set(customer_writeset) - set(local_write_log)

For each missing write, the receiving branch contacts the branch that originally created the write ID, retrieves the complete write record, and applies it locally. The branch performs this synchronization before responding to a query or committing a new customer update.

This direct synchronization is important because normal propagation may be delayed. It enforces Read Your Writes without requiring the customer to wait until every replica has naturally received every update.

Replication and Idempotent Application

After a branch completes an original deposit or withdrawal, it propagates that write to peer branches. A branch stores the full operation record, including the write ID, interface name, and amount. Before applying a received write, the branch checks whether the write ID already exists in its write log. This makes applying writes idempotent: duplicate propagation or synchronization requests do not modify the balance twice.

Output Generation

The client processes customer events in ascending event-ID order and writes successful responses to output.json. Each output event includes the customer ID and the result returned for that event. Query results include the branch contacted and the balance observed by the customer. Deposit and withdrawal results record a successful operation and destination branch.

This output structure follows the provided project example and makes each event result easy to evaluate independently.

Results

The completed implementation produced an output.json file with 60 customer event records from the local verification input. The output included successful deposits, withdrawals, and queries across multiple branch replicas.

The local verification scenario contained 19 cross-branch queries. Each query returned a balance that included the requesting customer's earlier successful writes, even when the query was sent to a branch different from the one that accepted the original write. For example, after a customer deposited $10 through Branch 1, a later query through Branch 2 returned a balance of $10 rather than the stale value of $0.

Additional direct synchronization testing confirmed the core Read Your Writes behavior. A write created at Branch 1 was intentionally absent from Branch 2's local log. When Branch 2 received a customer request containing that write ID, it retrieved the missing write from Branch 1, applied it once, and then processed the customer request using the updated balance. In a deposit-$400-at-Branch-1 followed by withdrawal-$400-at-Branch-2 test, both branches correctly ended with a balance of $0.

These results demonstrate that the system enforces Read Your Writes consistency: a customer's writeset identifies the updates the customer must observe, and a branch synchronizes missing writes before serving that customer's later request. The implementation therefore avoids stale read behavior while preserving the original gRPC project's distributed branch architecture.